Database Schema

Overview

The application uses a SQLite database with three main tables:

- `food_entries`: food item records shown on the public site and managed in the admin UI
- `users`: Google-authenticated users known to the system
- `sessions`: signed login sessions tied to users

The current database file is local in development and volume-backed in production.

Table: `food_entries`

Primary application data lives in `food_entries`.

Columns

Column	Type	Notes
`id`	`INTEGER`	Primary key, autoincrement
`name`	`TEXT`	Public food item name
`sustainability_index`	`REAL`	Derived score: nutrition composite + environmental composite
`tagged_recipes`	`TEXT`	Stored as a JSON array string
`created_at`	`TEXT`	ISO timestamp
`updated_at`	`TEXT`	ISO timestamp
`protein`	`REAL`	Raw nutrition input
`fiber`	`REAL`	Raw nutrition input
`vitamin_a`	`REAL`	Raw nutrition input
`vitamin_c`	`REAL`	Raw nutrition input
`vitamin_e`	`REAL`	Raw nutrition input
`calcium`	`REAL`	Raw nutrition input
`iron`	`REAL`	Raw nutrition input
`magnesium`	`REAL`	Raw nutrition input
`potassium`	`REAL`	Raw nutrition input
`saturated_fat`	`REAL`	Raw nutrition input
`added_sugar`	`REAL`	Raw nutrition input
`sodium`	`REAL`	Raw nutrition input
`nutrient_rich_food_index`	`REAL`	Derived from nutrition inputs
`nutrition_composite_score`	`REAL`	Derived from NRFI thresholds

Column	Type	Notes
freshwater_withdrawals	REAL	Environmental indicator 2-1
stress_weighted_water_use	REAL	Environmental indicator 2-2
acidifying_emissions	REAL	Environmental indicator 2-3
eutrophying_emissions	REAL	Environmental indicator 2-4
ghg_emissions	REAL	Environmental indicator 2-5
land_use	REAL	Environmental indicator 2-6
environmental_composite_score	REAL	Derived from scored environmental indicators

Indexes

- idx_food_entries_name on name

Important Implementation Detail

tagged_recipes is stored in SQLite as TEXT, but the application treats it as a JSON array. In API responses it is parsed into a normal JavaScript array.

Table: users

Stores local user records associated with Google authentication.

Columns

Column	Type	Notes
id	INTEGER	Primary key, autoincrement
google_sub	TEXT	Google subject identifier, unique
email	TEXT	User email, unique
name	TEXT	Display name
picture	TEXT	Optional profile image URL
created_at	TEXT	ISO timestamp
updated_at	TEXT	ISO timestamp
last_login_at	TEXT	ISO timestamp

Indexes

- idx_users_email on email

Table: `sessions`

Stores server-side login sessions.

Columns

Column	Type	Notes
`id`	`TEXT`	Session token identifier, primary key
`user_id`	`INTEGER`	Foreign key to `users.id`
`expires_at`	`TEXT`	ISO timestamp
`created_at`	`TEXT`	ISO timestamp

Indexes and Constraints

- `idx_sessions_user_id` on `user_id`
- foreign key: `user_id REFERENCES users(id) ON DELETE CASCADE`

Example `food_entries` Rows

These examples are drawn from the current local database and show the shape of real records.

id	name	tagged…	protein	fiber	saturated…	sodium	[illegible]	freshwater…	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	nutrient_rich_food_index	sustainability_index	…_composite_score
9	Lentil Chili	["lentil chili","vegan entree","hot bar"]	18.0	14.0	0.6	540.0	180.0	5200.0	8.4	6.8	1.3	2.4	180.1857	5.0	4.5	9.5
10	Grilled Chicken Bowl	["grilled chicken","bowl","rice bowl"]	32.0	5.0	1.9	610.0	410.0	9600.0	14.7	9.8	2.6	4.8	123.5643	6.0	3.1667	7.1667
11	Roasted Broccoli	["roasted broccoli","vegetable side","green veggie"]	4.0	6.0	0.1	180.0	95.0	2400.0	0.8	3.2	0.7	1.2	203.7544	4.0	5.0	10.0

Derived-Field Notes

Several columns in `food_entries` are not freeform data-entry fields. They are calculated by the server:

- `nutrient_rich_food_index`
- `nutrition_composite_score`
- `environmental_composite_score`
- `sustainability_index`

The frontend may preview these values live, but the backend is the source of truth.

Environmental Scoring Notes

The environmental raw inputs are stored directly:

- `freshwater_withdrawals`
- `stress_weighted_water_use`
- `acidifying_emissions`
- `eutrophying_emissions`
- `ghg_emissions`
- `land_use`

These inputs are first converted into 1-to-5 scores by threshold rules, and then averaged to produce `environmental_composite_score`.

The UI also derives these additional display-only factor scores at response/render time:

- `water_use_score`
- `nitrogen_use_score`
- `carbon_use_score`
- `land_use_score`

These are not currently stored as database columns.

Operational Notes

- The app currently depends on the `sqlite3` CLI in addition to the SQLite database file.
- In development, the database is typically `data.sqlite`.
- In production on Railway, the database should live at the configured `DB_PATH` on a persistent volume.
- `data.sqlite` is now treated as local-only state and should not be committed as production data.

Practical Things To Know

- `food_entries.name` is the main human-readable identifier used across the UI.
- Search is currently centered on `name` and `tagged_recipes`.
- If you ever need to migrate the schema, do it carefully: the app does not yet have a dedicated migrations system.
- Timestamps are stored as text in ISO-like UTC strings, which keeps them easy to serialize and display.